

ENGE707 • WEEK 1

Foundations of Data Engineering & AI

From zero to your first working Python pipeline — installing Python, writing your first code, and thinking like a data engineer.



What you'll be able to do by the end of today

- Explain how data engineering supports AI systems
- Install Python, VS Code, and set up a project environment.
- Write basic Python: variables, lists, dictionaries, conditionals, loops and functions
- Read a CSV file and manipulate table data with Pandas
- Use NumPy to combine numerical features
- Recognise classification, regression, clustering and generative AI problems
- Understand why data quality can matter more than model complexity

HANDS-ON MINDSET

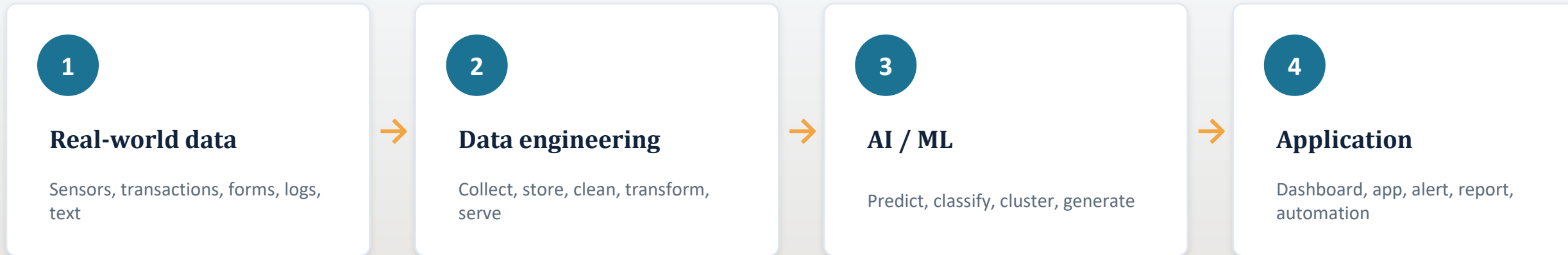
Run code. Break it. Change thresholds. Inspect mistakes.

That is how AI systems are actually learned — not by reading about them.

Today's classroom dataset is a small wound-measurement CSV — just one practice example. The pattern applies to banking, retail, transport and more.

THE BIG PICTURE

AI is not just a model — it's a workflow



A common beginner mistake: starting with “which model should I use?”

A better systems question: do we have the right data, the right labels, the right workflow, and a way to learn from mistakes?

BLOCK 1 OF 4: REAL-WORLD DATA

What does “real-world data” actually look like?

IN TODAY'S EXAMPLE

A nurse measures a patient's temperature and pH at the bedside and writes it down. That single bedside reading becomes one row in `wound_data.csv`:

```
patient_id=9, temperature=37.5,  
pH=7.50, status=deteriorating
```

Before it was a row in a spreadsheet, it was just a number on a chart at someone's bedside.

ACROSS OTHER DOMAINS

- Banking: a customer taps their card — a transaction record is created
- Retail: a barcode scan at checkout — a line item is logged
- Manufacturing: a vibration sensor reports a reading every second
- Social media: someone posts a comment — text and metadata are captured

What is data? From data to information to knowledge

1

Data

Raw, elementary facts — on their own they don't yet mean much

e.g. temperature = 36.4

2

Information

Data that has been organised so it has meaning

e.g. "average temperature is rising over the week"

3

Knowledge

Processed information that supports a decision

e.g. "rising temperature + rising pH predicts deterioration"

A single number is data. A trend across many rows is information. A rule that predicts an outcome is knowledge.

WHAT KIND OF DATA ARE WE USING?

Types of data: structured, semi-structured, unstructured

Structured

Fixed format — rows and columns, a known schema

e.g. a CSV file, a database table

Semi-structured

Some organisation, but no fixed table schema

e.g. JSON, XML

Unstructured

No predefined format at all

e.g. free text, images, audio, video

The dataset you choose should be structured, tabular, alpha-numeric data — image datasets won't work well with the techniques this course covers.

BLOCK 2 OF 4: DATA ENGINEERING

What does “data engineering” actually look like?

IN TODAY'S EXAMPLE

Every pandas line you write this lecture is a data engineering step — just a small one:

```
df = pd.read_csv("wound_data.csv") # collect

df.isna().sum()                    # inspect

df["combined_risk"] = (            # transform
    0.65 * (df["temperature"] - 36.5)
    + 0.35 * (df["pH"] - 7.0) * 2
)
```

ACROSS OTHER DOMAINS

- Banking: a nightly job pulls transactions from a dozen branch databases into one warehouse
- Retail: point-of-sale data is merged with online orders and de-duplicated
- Manufacturing: sensor streams are filtered for noise before they're stored
- Healthcare: patient records from different systems are reconciled into one file

Every domain runs the same steps — collect, clean, transform, store — just on different data.

Why data is hard to work with: the 4 Vs

1 Volume

How much data is generated

e.g. gigabytes to petabytes of transactions, logs, sensor readings

2 Velocity

How fast data is produced

e.g. streaming sensor feeds, live transactions, social media posts

3 Variety

How many different forms data takes

e.g. tables, text, images, video, JSON, sensor streams

4 Veracity

How correct and trustworthy the data is

e.g. the most important V — bad data gives bad answers, however big or fast

BLOCK 3 OF 4: AI / ML

What does the “AI / ML” block actually look like?

IN TODAY'S EXAMPLE

The classification rule you'll write later today is the whole “AI/ML” block, in miniature:

```
df["prediction"] = np.where(
    (df["temperature"] > 37.7)
    & (df["pH"] > 7.65),
    "deteriorating", "stable"
)
```

Features in (temperature, pH) → a prediction out (stable / deteriorating).

ACROSS OTHER DOMAINS

- Banking: transaction features in → fraud / not fraud out
- Retail: browsing history in → a recommended product out
- Manufacturing: sensor readings in → a failure-risk score out
- Social media: post text in → spam / not spam out

Types of AI problems

Your problem statement needs to name one of these.

1 Classification

Predict a category.

e.g. spam / not spam, at-risk / not at-risk

2 Regression

Predict a number.

e.g. price, demand, healing days

3 Clustering

Find groups without labels.

e.g. customer segments, machine states

4 Generative AI

Create text, images, code, or summaries.

e.g. report drafts, chatbots

BLOCK 4 OF 4: APPLICATION

What does “application” actually look like?

IN TODAY'S EXAMPLE

In a real hospital system, our “deteriorating” prediction wouldn't stop at a notebook — it would show up as an alert:

⚠ Patient 9 — deteriorating risk
Please review at bedside

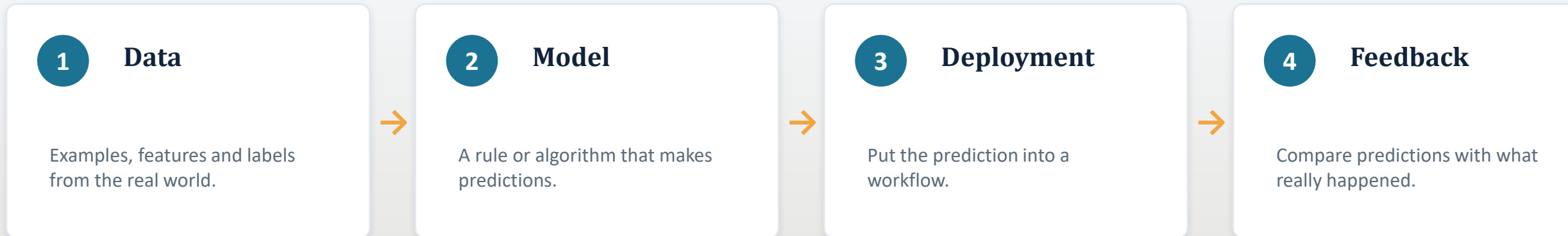
This is a mock-up of a tablet alert — not a real hospital system.

ACROSS OTHER DOMAINS

- Banking: a fraud alert pops up and blocks the transaction
- Retail: a “recommended for you” carousel updates on the homepage
- Manufacturing: a maintenance ticket is auto-created before a machine fails
- Social media: a flagged post is hidden pending review

FOUNDATIONS

The AI pipeline: data → model → deployment → feedback



EXAMPLE · FRAUD DETECTION

Transactions → fraud model → bank alert → customer confirms/denies → model and rules improve.

PART I

Getting Python running on your computer

Assume nothing is installed yet. We build up from a completely blank machine.



BEFORE WE INSTALL ANYTHING

What is a programming language? What is Python?

A PROGRAMMING LANGUAGE

A precise way of writing instructions for a computer to follow — like a recipe, but every step must be exact.

“Add a pinch of salt” works for a human cook. A computer needs “add 2 grams of salt” — no guessing.

PYTHON

One of the most widely used programming languages for data, AI and automation — readable, beginner-friendly, and free.

PYTHON IS THE LANGUAGE, NOT A FRAMEWORK

A framework is a bigger ready-made structure built using a programming language — e.g. Django for web apps, or PyTorch for machine learning.

In this course, Python is the language. NumPy, Pandas, Matplotlib, scikit-learn and Jupyter are packages — toolboxes we use inside Python.

Today's goal: get Python and its tools installed and talking to each other.

STEP 1 OF 6

Install Python

- 1 Go to python.org/downloads in your web browser.
- 2 Click the yellow “Download Python 3” button — it detects your operating system automatically.
- 3 Open the downloaded installer.
- 4 Windows only: tick “Add Python to PATH” before clicking Install. This lets your computer find Python from the terminal.
- 5 Click Install (or Install Now) and wait for it to finish.

⚠ “Add Python to PATH” is the single most common step students forget — without it, Windows can't find Python from the terminal.

MAC USERS

There is no “Add to PATH” checkbox — the macOS installer handles this automatically. Just run the .pkg installer and follow the prompts.

WHAT IS “PATH”?

A system setting that tells the terminal where to find installed programs, such as Python.

STEP 2 OF 6

What is a terminal, and did Python install correctly?

WHAT IS A TERMINAL?

A text-based window where you type commands instead of clicking icons. It's how we talk directly to the computer.

Think of it as a chat window with your computer — you type a command, press Enter, and it replies.

We will mostly use the terminal built into VS Code (next step) — no need to hunt for a separate app.

VERIFY THE INSTALL

```
python --version  
# on Windows, if that fails, try:  
py --version
```

Expect to see something like Python 3.12.4. If you instead get “command not found”, Python was not added to PATH — re-run the installer and tick that box, or restart your computer.

STEP 3 OF 6

Install VS Code — the editor we'll write code in

WHAT IS AN IDE / CODE EDITOR?

A program for writing and running code — like Microsoft Word, but for code instead of essays. VS Code (Visual Studio Code) is free and made by Microsoft.

- Go to code.visualstudio.com
- Download the installer for your operating system
- Run it and accept the default options
- Open VS Code once installation finishes

INSTALL THESE EXTENSIONS

Open the Extensions panel: Ctrl + Shift + X, then search and install each:

Python

Runs and checks Python code; selects interpreters.

Jupyter

Runs notebook cells; connects notebooks to a Python kernel.

GitHub Copilot

Optional — gives code suggestions and explanations.

STEP 4 OF 6

Organise your work: create a project folder

- Create a new folder anywhere convenient, e.g. Documents.
- Suggested name: data_ai_foundations — no spaces, easy to type.
- In VS Code: File → Open Folder → select data_ai_foundations → Open.
- Everything for this course lives inside this one folder: notebooks, CSV files, and the environment we build next.

WHY ONE FOLDER MATTERS

Data engineering is as much about organisation as code. A single, well-named project folder means:

- Nothing gets lost across folders
- Your CSV files sit right beside your notebook
- Anyone (including future-you) can find everything

```
data_ai_foundations  
(empty for now — we fill it in next)
```

STEP 5 OF 6

Create a private Python environment

WHAT IS A VIRTUAL ENVIRONMENT?

A private, isolated Python space for one project — like giving each project its own toolbox, so tools from one project never get mixed up with another.

In the VS Code terminal, inside your project folder:

```
python -m venv .venv

# Activate it (Windows PowerShell):
.venv\Scripts\activate

# You should now see (.venv) at
# the start of the terminal line.
```

READING THE COMMAND

python	Uses Python.
-m	Runs a Python module as a command.
venv	Python's built-in virtual-environment tool.
.venv	Folder name for this project's private environment.

If PowerShell blocks activation, run this once, then activate again:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope
CurrentUser
```

STEP 6 OF 6

Packages: install the tools for this course

Packages are add-on toolboxes written by other developers. Python gives the language; packages give extra tools for tables, arrays, plots and machine learning.

```
pip install numpy pandas scikit-learn  
matplotlib jupyter
```

Run this inside the activated (.venv) terminal.

- **numpy** fast maths + arrays
- **pandas** CSV files + tables
- **matplotlib** graphs and plots
- **scikit-learn** machine learning models
- **jupyter** notebook interface

WHAT IF **pip install** FAILS INSIDE A NOTEBOOK?

```
%pip install numpy pandas scikit-learn\  
matplotlib jupyter
```

Run this inside a notebook cell (with the % sign), then restart the kernel and try importing again.

Takeaway: activate the environment first, then install packages, then select the same environment as your notebook kernel.

Jupyter notebooks: connect and run your first cell

NOTEBOOK vs KERNEL

The notebook is the page you write on. The kernel is the Python engine running behind it. The ipykernel package is the bridge that connects your .venv to the notebook.

- File → New File → Jupyter Notebook
- Save it as lecture1_coding.ipynb inside data_ai_foundations
- Click “Select Kernel” (top right) → choose Python (.venv)
- Type `print("Python is working")` in the first cell and press Shift+Enter to run it

```
.\.venv\Scripts\python.exe -m pip install ipykernel

.\.venv\Scripts\python.exe -m ipykernel install
  --user --name data_ai_foundations
  --display-name "Python (.venv)"
```

```
print("Python is working")
```

```
# Expected output:
# Python is working
```

YOUR FIRST REAL PYTHON

Python basics: variables & data types

A variable is a named box that stores a value. Python figures out the type automatically.

```
patient_id = 1           # int (whole number)
temperature = 36.7        # float (decimal)
status = "stable"         # string (text)
is_high_risk = False      # boolean
(True/False)

print(patient_id, temperature, status)
```

int

1, 20, -5

float

36.7, 7.08

str

"stable"

Why this matters for data engineering: every column in a CSV file is really just a long list of these same basic types — that's what Pandas is built on top of.

YOUR FIRST REAL PYTHON

Python basics: lists, dictionaries, conditionals & loops

LISTS — an ordered collection

```
temperatures = [36.4, 36.6, 37.5, 38.1]
print(temperatures[0])    # 36.4 (first item)
```

DICTIONARIES — labelled key/value pairs

```
patient = {"id": 1, "temperature": 36.4,
           "status": "stable"}
print(patient["temperature"])    # 36.4
```

IF / ELSE — conditional logic

```
if temperature > 37.7:
    print("high")
else:
    print("normal")
```

FOR LOOPS — repeat for every item

```
temperatures = [36.5, 37.2, 38.1, 39.0, 36.9]
for t in temperatures:
    print(t, "is high" if t > 37.7 else "is
normal")
```

These four building blocks power almost every line of pandas/numpy code you'll write this semester.

YOUR FIRST REAL PYTHON

Python basics: functions

A function packages up reusable logic so you don't repeat yourself. `def` defines it; `return` sends back a result.

```
def classify_status(temperature, ph):  
    if temperature > 37.7 and ph > 7.65:  
        return "deteriorating"  
    else:  
        return "stable"  
  
result = classify_status(38.0, 7.8)  
print(result)    # deteriorating
```

READING THE PIECES

<code>def</code>	starts a new function definition
<code>classify_status(...)</code>	the function's name and inputs
<code>if / else</code>	decision logic inside the function
<code>return</code>	sends the answer back to whoever called it

This is exactly the pattern the labs use for classification rules — you'll write these yourself.

WHEN SOMETHING GOES WRONG

Common setup errors & fast fixes

PowerShell blocks activation

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser, then activate again.

.venv not in Select Kernel list

Install ipykernel, register it, then fully restart VS Code.

No module named pandas/numpy

Run %pip install ... inside a notebook cell, then restart the kernel.

FileNotFoundError for the CSV

Check the CSV sits beside the notebook, not inside .venv, and the filename is exact.

NameError: df is not defined

Run cells in order — later cells depend on variables from earlier ones.

Plot does not appear

Run %matplotlib inline once near the top of the notebook.

PART II

From CSV to your first AI predictions

Now that Python is running, let's put it to work on real tabular data.



BEFORE WE TOUCH ANY CODE

Data handling fundamentals

Row

One observation

e.g. one patient visit, one sale

Column

One type of information

e.g. temperature, price, age

Feature

Input used for prediction

e.g. temperature and pH

Label

The answer we train/test against

e.g. stable / deteriorating

Data quality

Missing, wrong, duplicated or biased values

Data manipulation

Filter rows, create columns, save outputs

OUR PRACTICE DATASET

Create wound_data.csv

CSV = comma-separated values. A simple text file that stores table data — the first line is the header row.

```
patient_id,temperature,pH,status
1,36.4,7.08,stable
2,36.6,7.01,stable
3,36.7,7.24,stable
4,36.9,7.33,stable
5,37.0,7.05,stable
6,37.1,7.18,stable
7,37.2,7.31,stable
8,37.4,7.42,stable
9,37.5,7.50,deteriorating
10,37.6,7.53,stable
```

HOW TO CREATE IT IN VS CODE

- Right-click the project folder → New File
- Name it exactly wound_data.csv
- Paste the CSV text
- Save with Ctrl + S

This dataset is intentionally small and imperfect — a simple rule gets ~75% accuracy, so we can inspect real mistakes.

FIRST DATA ENGINEERING STEP

Read the CSV file with Pandas

```
import pandas as pd

df = pd.read_csv("wound_data.csv")
df.head()
```

<code>import pandas as pd</code>	brings in the table-handling toolbox
<code>pd.read_csv(...)</code>	reads the CSV file from the project folder
<code>df</code>	Is a variable name that stores the table as a DataFrame
<code>df.head()</code>	shows the first five rows

INSPECT BEFORE YOU TRUST

```
df.shape      # rows and columns
df.columns    # column names
df.info()     # dtypes + missing values
df.describe() # simple statistics
```

Before using AI, inspect the file. Do not trust data blindly. Many AI problems come from missing values, wrong columns, wrong data types, or duplicated rows.

NUMERICAL FOUNDATIONS

NumPy: Python's number-handling library

```
import numpy as np

readings = np.array([36.5, 37.2, 38.1])

print(readings.mean())
print(readings > 37.7)
```

- NumPy is a Python library for fast numerical calculations
- Its main object is an array: a list or grid of numbers
- It lets us calculate on many values at once

WHY WE USE IT

Most real-world data becomes numbers before analysis. NumPy helps you store, compare, summarise and transform those numbers efficiently.

array



```
[ 36.5  37.2  38.1 ]
```

```
mean = 37.3
```

Instead of looping manually, NumPy can apply one operation across the whole array.

NumPy: arrays and the correlation check

```
import numpy as np

temp = df["temperature"].to_numpy()
ph = df["pH"].to_numpy()

correlation = np.corrcoef(temp, ph)[0, 1]
print("Correlation:", round(correlation, 2))
```

- `to_numpy()` turns a Pandas column into a NumPy array
- `np.corrcoef()` checks how two variables move together

WHY THIS MATTERS

In this dataset, temperature and pH are interdependent — they tend to move together, but not perfectly.

That means one feature alone is not enough to explain every row — a key idea we'll use again in feature engineering.

`temp[:5]` previews the first five values — Python counts from zero, so slicing is a common way to peek at data.

CREATING NEW COLUMNS

Make new features from temperature and pH

```
df["high_temperature"] = df["temperature"] > 37.7
df["high_pH"] = df["pH"] > 7.65

df["combined_risk"] = (
    0.65 * (df["temperature"] - 36.5)
    + 0.35 * (df["pH"] - 7.0) * 2
)
```

- Two True/False columns are created using simple thresholds
- combined_risk blends both variables into one practice feature (not a clinical score)
- This is why data manipulation matters: AI models don't work directly from messy raw data — they need shaped, meaningful features

PREDICTING A CATEGORY

Classification: a two-condition prediction rule

```
df["prediction"] = np.where(
    (df["temperature"] > 37.7)
    & (df["pH"] > 7.65),
    "deteriorating", "stable"
)
df["correct"] = df["prediction"] == df["status"]
accuracy = df["correct"].mean()
print("Accuracy:", round(accuracy, 2))
# Save the updated table as a new CSV file
df.to_csv("wound_data_with_predictions.csv",
index=False)
```

- Classification predicts a category: stable or deteriorating
- & means both conditions must be true
- The rule is intentionally imperfect — accuracy is not 100%

WHY ACCURACY ISN'T ENOUGH

```
mistakes = df[df["correct"] == False]
mistakes[["patient_id", "temperature",
          "pH", "status", "prediction"]]
```

False positive: predicted deteriorating, actually stable.

False negative: predicted stable, actually deteriorating.

Mistakes are useful — they tell us what the data or rule needs fixing, not just how “good” the model is.

Save, append, drop, and edit a CSV

```
# Open the CSV that was already saved
df = pd.read_csv("wound_data_with_predictions.csv")

# Create a new column
new_column = pd.DataFrame({
    "risk_level": np.where(
        (df["temperature"] > 37.7) & (df["pH"] > 7.65),
        "high",
        "low"
    )
})

# Append the new column to the DataFrame
df = pd.concat([df, new_column], axis=1)

# Drop a column
df = df.drop(columns=["correct"])

# Remove one value by replacing it with a missing value
df.loc[df["patient_id"] == 9, "pH"] = np.nan

# Save the final updated version
df.to_csv("wound_data_updated.csv", index=False)

print(df)
```

axis=0 → place data underneath → add rows

axis=1 → place data beside → add columns

What each step does

Open: loads the previously saved CSV into a Pandas Data Frame.

Create: generates a new risk_level column containing "high" or "low".

Append: adds the new column horizontally using pd.concat(..., axis=1).

Drop: removes the entire correct column from the table.

Remove a value: replaces patient 9's pH value with np.nan, meaning missing.

PREDICTING A NUMBER

Regression: predicting healing_days

```
from sklearn.linear_model import LinearRegression

X = df[["temperature", "pH"]]
y = df["healing_days"]

model = LinearRegression()
model.fit(X, y)
model.predict([[38.2, 7.8]])
```

Based on the existing dataset, the model is being asked how many healing days would you predict for a patient with a temperature of 38.2°C and a pH of 7.8?

CLASSIFICATION vs REGRESSION

Regression is not asking “which class is this?” It is asking “what number should this be?”

- House price prediction → price
- Energy demand forecasting → demand value
- Student performance estimation → final mark
- Lecture dataset → healing_days

Clustering: grouping without using the label

```
from sklearn.cluster import KMeans

X = df[["temperature", "pH"]]
kmeans = KMeans(n_clusters=3, random_state=7)
df["cluster"] = kmeans.fit_predict(X)
print(df[["patient_id", "temperature", "pH",
"cluster"]])
```

- Clustering is unsupervised — it never sees the status column
- Useful for customer groups, machine states, sensor patterns
- K-means (a class within the library sklearn) is a machine-learning method that automatically divides data into K groups of similar points. KMeans(n_clusters=3), so the algorithm tries to create three groups of patients based on temperature and pH.
- The number 7 in random 7 has no special meaning. You could use 1, 42, 100, or another integer. Using the same number makes the random starting process repeatable, so the same data and settings should produce the same clustering each time

CLUSTERS ARE NOT AUTOMATICALLY “TRUTH”

The algorithm finds statistical groups — a human still has to interpret what each group actually means in the real world.

Examples across domains

- Retail: customers grouped by spending and visits
- Manufacturing: machine states from sensor patterns

CREATING CONTENT FROM DATA

Generative AI: turning a row into a draft summary

```
row = df.iloc[8]
prompt = f"""
Write a short non-diagnostic summary:
Patient ID: {row['patient_id']}
Temperature: {row['temperature']}
pH: {row['pH']}
Prediction: {row['prediction']}
Keep it factual and mention uncertainty.
"""
print(prompt)
```

WHAT THIS CODE DOES NOT DO

It does not call a generative AI model — it only prepares a structured prompt that could be given to one.

`f"""..."""` builds a multi-line text string; values in `{braces}` are pulled in from your Python variables.

Important: generated text is not automatically true. It must be grounded in data, checked for accuracy, and reviewed by a human in sensitive contexts.

BETTER DATA BEATS A FANCIER MODEL

Data-centric AI: check the data before blaming the model

```
df[df["correct"] == False]    # rule failures
df.isna().sum()               # missing values
df["status"].value_counts()   # class balance
df.duplicated().sum()         # duplicate rows
```

QUESTIONS TO ALWAYS ASK

- Are labels consistent between reviewers?
- Are measurements collected the same way?
- Are classes balanced, or is one under-represented?
- Are borderline cases labelled too confidently?
- Are missing values hiding a pattern?

Better data often beats a fancier model.

BETTER DATA BEATS A FANCIER MODEL

Data-centric AI: check the data before blaming the model

```
df[df["correct"] == False]    # rule failures
df.isna().sum()               # missing values
df["status"].value_counts()   # class balance
df.duplicated().sum()         # duplicate rows
```

QUESTIONS TO ALWAYS ASK

- Are labels consistent between reviewers?
- Are measurements collected the same way?
- Are classes balanced, or is one under-represented?
- Are borderline cases labelled too confidently?
- Are missing values hiding a pattern?

Better data often beats a fancier model.

Rule failures: create and inspect one wrong case

```
df = pd.read_csv("wound_data_with_predictions.csv")
df_rulefail = df.copy()

# Deliberately create one prediction that does not follow the
rule
df_rulefail.loc[
    df_rulefail["patient_id"] == 1,
    "prediction"
] = "deteriorating"

# Calculate what the prediction should be according to the
rule
df_rulefail["rule_prediction"] = np.where(
    (df_rulefail["temperature"] > 37.7)
    & (df_rulefail["pH"] > 7.65),
    "deteriorating",
    "stable"
)

# Compare the stored prediction with the rule-based
prediction
df_rulefail["follows_rule"] = (
    df_rulefail["prediction"]
    == df_rulefail["rule_prediction"]
)

# Display only rule failures
df_rulefail[df_rulefail["follows_rule"] == False]
```

WHEN YOU RUN IT

patient_id	status	rule_prediction	follows_rule
1	stable	deteriorating	False

The rule predicted deteriorating , but the condition for temperature and pH used should have produced stable. This row should be inspected before blaming the model.

CHECK 2 · MISSING VALUES

Missing values: create one blank cell and count it

```
df = pd.read_csv("wound_data_with_predictions.csv")

# create a copy and remove one pH value
df_missing = df.copy()
df_missing.loc[df_missing["patient_id"] == 2,
               "pH"] = np.nan

df_missing.info()
```

Key idea: missing data can hide a pattern.

WHEN YOU RUN IT

```
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   patient_id   10 non-null    int64
1   temperature  10 non-null    float64
2   pH           9 non-null     float64
3   status        10 non-null    str
4   prediction    10 non-null    str
5   correct       10 non-null    bool
dtypes: bool(1), float64(2), int64(1), str(2)
memory usage: 542.0 bytes
```

The 1 null (or 9 non-null instead of 10) beside the pH means one pH value is missing.

Missing values may reflect collection problems or clinical workflow.

CHECK 3 · CLASS BALANCE

Class balance: see whether one category dominates

```
df_balance = df.copy()

# Save the already imbalanced dataset
df_balance.to_csv(
    "wound_data_imbalanced.csv",
    index=False
)

# Display the number of patients in each class
print(df_balance["status"].value_counts())
```

WHEN YOU RUN IT

```
status
stable      9
deteriorating 1
Name: count, dtype: int64
```

There are many more stable rows than deteriorating rows. Accuracy can look good even when the minority class is weak.

Key idea: class imbalance can make accuracy misleading.

Duplicate rows: create one repeated patient record

```
df = pd.read_csv("wound_data_with_predictions.csv")

# create a copy and duplicate the first row
df_dupes = df.copy()
df_dupes = pd.concat(
    [df_dupes, df_dupes.iloc[[0]]],
    ignore_index=True
)

df_dupes.to_csv("wound_data_duplicates.csv", index=False)

# count exact duplicate rows
df_dupes.duplicated().sum()
```

`ignore_index=True` tells Pandas to discard the old row numbers and create fresh row numbers after combining DataFrames. Without it, the copied row keeps its original index:

0 1 2 ... 9 0 ← duplicated index

With `ignore_index=True`, Pandas renumbers everything:

0 1 2 ... 9 10

WHEN YOU RUN IT

1

`np.int64(1)`

The output 1 means one row is an exact duplicate. Duplicates can make the dataset look larger than it is.

SUMMARY

What Week 1 covered

- Python is the language; VS Code is the editor; .venv is your private environment
- pip installs packages; Jupyter notebooks need a kernel to run cells
- Variables, lists, dictionaries, conditionals, loops and functions are Python's building blocks
- Pandas reads and manipulates table data; NumPy handles numerical arrays and calculations
- Classification predicts a category; regression predicts a number; clustering finds groups; generative AI creates content
- Data-centric AI improves the data and labels before reaching for a fancier model

NEXT WEEK

Data representation, storage formats, ETL/ELT pipelines, and batch vs. streaming processing.

Keep your `data_ai_foundations` folder and `.venv` — we build on it directly.